Árboles de Decisión

Aprendizaje Supervisado · CART, Gini y Entropía · Riesgo Crediticio

Inteligencia Artificial · Sexto Semestre · Ingeniería de Sistemas de Información · PUCE

Objetivos de Aprendizaje

Al finalizar esta clase serás capaz de:

1. Explicar cómo un árbol clasifica con preguntas Sí/No y cómo CART elige cada división con Gini o Entropía.
2. Reconocer el sobreajuste de un árbol y controlarlo con max_depth, min_samples_leaf y GridSearchCV.
3. Entrenar, visualizar e interpretar un DecisionTreeClassifier: reglas e importancia de variables.
4. Reentrenar el modelo final con el 100% de los datos, exportarlo con joblib y usarlo con datos nuevos.

¿Qué es un Árbol de Decisión?

Un modelo que clasifica haciendo preguntas de Sí/No sobre los datos

Definición: modelo de aprendizaje supervisado que predice la clase de un caso nuevo recorriendo una jerarquía de preguntas binarias (Sí/No) sobre sus variables.

En lenguaje simple

- El árbol aprende qué preguntar y en qué orden.
- Cada pregunta divide los datos en grupos más parecidos entre sí.
- Al final de cada camino hay una clase: la más frecuente del grupo.

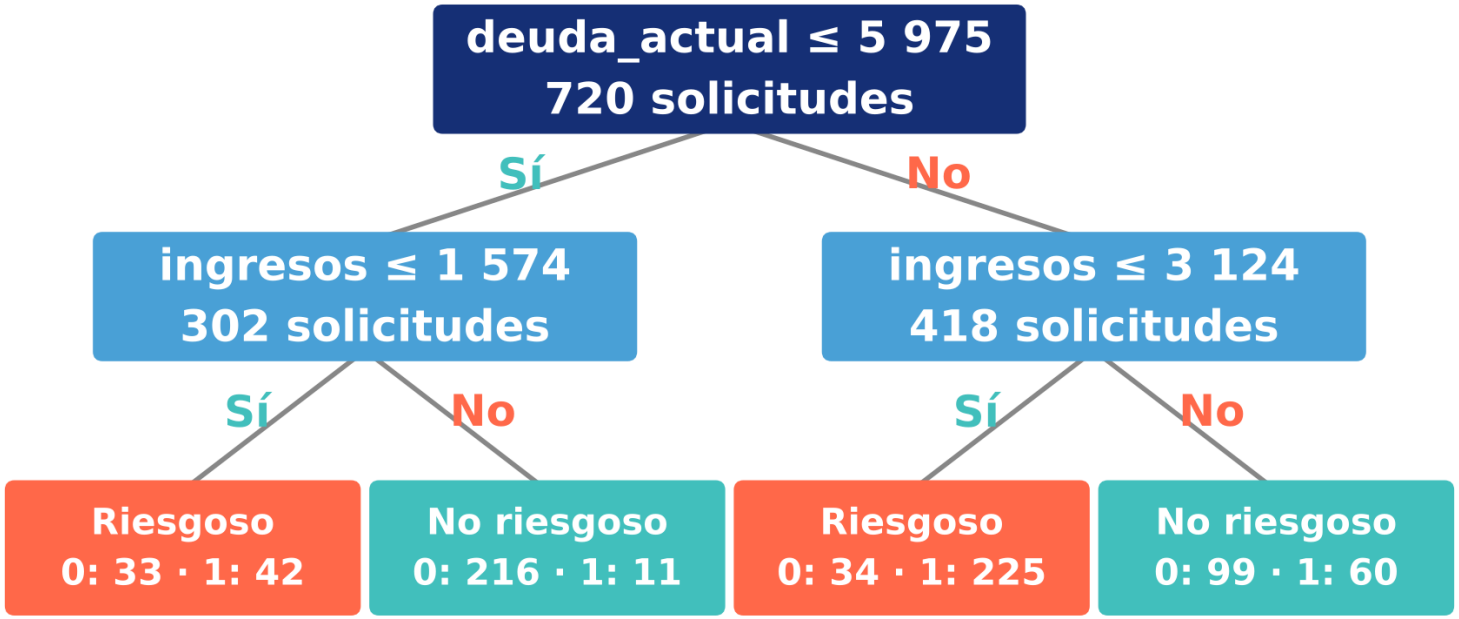
Analogía: un oficial de crédito

- ¿La deuda supera los $6 000?
- Si la respuesta es Sí: ¿gana más de $3 100?
- Según las respuestas: «riesgoso» o «no riesgoso».

Cada camino raíz → hoja es una regla SI … ENTONCES … que cualquier persona puede leer

Anatomía del Árbol

Árbol real de profundidad 2 aprendido con aprobacion_credito_ecuador.csv



Hojas: cuántas solicitudes de entrenamiento llegaron de cada clase (0 = no riesgoso, 1 = riesgoso).

Partes del árbol

- **Raíz:** primera pregunta; recibe todos los datos.
- **Nodo interno:** pregunta intermedia.
- **Rama:** respuesta Sí (izq.) o No (der.).
- **Hoja:** predicción final (clase mayoritaria).
- **Profundidad:** niveles de preguntas (aquí 2).

¿Cómo Aprende el Árbol? Algoritmo CART

CART = Classification And Regression Trees: el algoritmo que usa scikit-learn

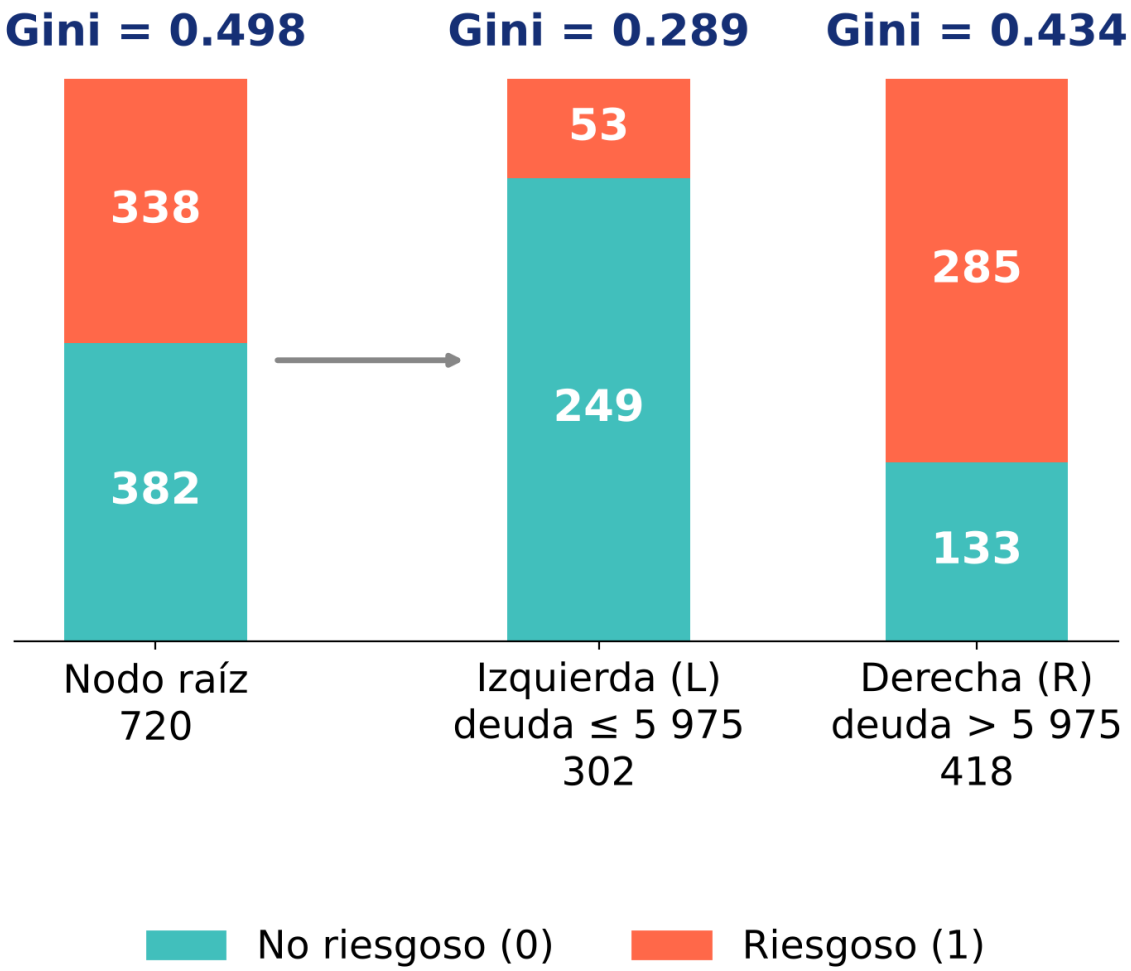
1	Empieza con todos los datos de entrenamiento en el nodo raíz.
2	Prueba TODAS las variables y TODOS los puntos de corte posibles.
3	Elige el corte que deja los nodos hijos más puros (mayor reducción de Gini).
4	Repite el proceso en cada hijo hasta cumplir una condición de parada.
5	Cada hoja predice la clase más frecuente entre sus casos.

CART es codicioso (greedy): elige la mejor división en cada paso, no el mejor árbol global

Fuente: Breiman, Friedman, Olshen y Stone (1984). Classification and Regression Trees. Wadsworth.

¿Qué es una Buena División?

Una división es buena cuando cada nodo hijo queda dominado por una sola clase



Primera división real

- Raíz: 382 vs 338 → muy mezclado.
- L: 249 vs 53 → casi todo No riesgoso.
- R: 133 vs 285 → mayoría Riesgoso.

Pureza: un nodo es puro cuando todos sus casos son de la misma clase. Se mide con Gini o con Entropía.

Criterio de División: Índice Gini

Mide la impureza de un nodo: qué tan mezcladas están las clases

Definición: probabilidad de clasificar mal un caso elegido al azar si se le asigna una clase según las proporciones del nodo.

Fórmula

$$\text{Gini}(t) = 1 - \sum_{k=1}^{K} p_k^2$$

p_k = proporción de la clase k en el nodo · K = número de clases

Rango y valor deseado

- Gini = 0 → nodo puro (deseado).
- Gini = 0.5 → máxima mezcla con 2 clases.
- Rango: 0 a 0.5 (2 clases) → más bajo es mejor.

criterion='gini' es el valor por defecto de DecisionTreeClassifier en scikit-learn

Gini Paso a Paso: el Nodo Raíz Real

720 solicitudes de entrenamiento: 382 no riesgosas (clase 0) y 338 riesgosas (clase 1)

Paso 1 — Proporciones de cada clase

$$p_0 = \frac{382}{720} = 0.531 \qquad p_1 = \frac{338}{720} = 0.469$$

Paso 2 — Aplicar la fórmula

$$\text{Gini(raíz)} = 1 - (0.531^2 + 0.469^2) = 0.498$$

Interpretación: 0.498 está casi en el máximo posible (0.5): la raíz está muy mezclada. El árbol necesita dividirla para obtener grupos más puros.

Gini de los Nodos Hijos y Gini Ponderado

División real: deuda_actual ≤ 5 975 (L = respuesta Sí · R = respuesta No)

Impureza de cada nodo hijo

$$\text{Gini}(L) = 1 - \left[\left(\frac{249}{302}\right)^2 + \left(\frac{53}{302}\right)^2\right] = 0.289$$

$$\text{Gini}(R) = 1 - \left[\left(\frac{133}{418}\right)^2 + \left(\frac{285}{418}\right)^2\right] = 0.434$$

Gini ponderado de la división (n = número de casos de cada nodo)

$$\text{Gini}_{\text{div}} = \frac{n_L}{n}\,\text{Gini}(L) + \frac{n_R}{n}\,\text{Gini}(R)$$

$$\text{Gini}_{\text{div}} = \frac{302}{720}(0.289) + \frac{418}{720}(0.434) = 0.373$$

Ganancia de Gini: Elegir la Mejor División

La mejor división es la que más reduce la impureza

$$\Delta\text{Gini} = \text{Gini(padre)} - \text{Gini}_{\text{div}}$$

Rango: 0 a Gini(padre) → más alto es mejor

Con los datos reales: $\Delta\text{Gini} = 0.498 - 0.373 = 0.125$

- CART calcula ΔGini para cada variable y para cada punto de corte posible.
- deuda_actual ≤ 5 975 obtuvo la mayor ganancia: por eso es la pregunta de la raíz.
- Los cortes se prueban en el punto medio entre dos valores consecutivos ordenados.
- En cada nodo hijo se repite el mismo cálculo con sus propios casos.

En cada nodo, CART elige la división con mayor ΔGini

Criterio de División: Entropía y Ganancia de Información

Alternativa a Gini, basada en la teoría de la información

Definición: mide la incertidumbre (desorden) de un nodo en bits; 0 = nodo puro.

Entropía de un nodo

$$H(t) = -\sum_{k=1}^{K} p_k \log_2(p_k)$$

Ganancia de Información (IG)

$$IG = H(\text{padre}) - \left[\frac{n_L}{n} H(L) + \frac{n_R}{n} H(R)\right]$$

Ejemplo con el nodo raíz real

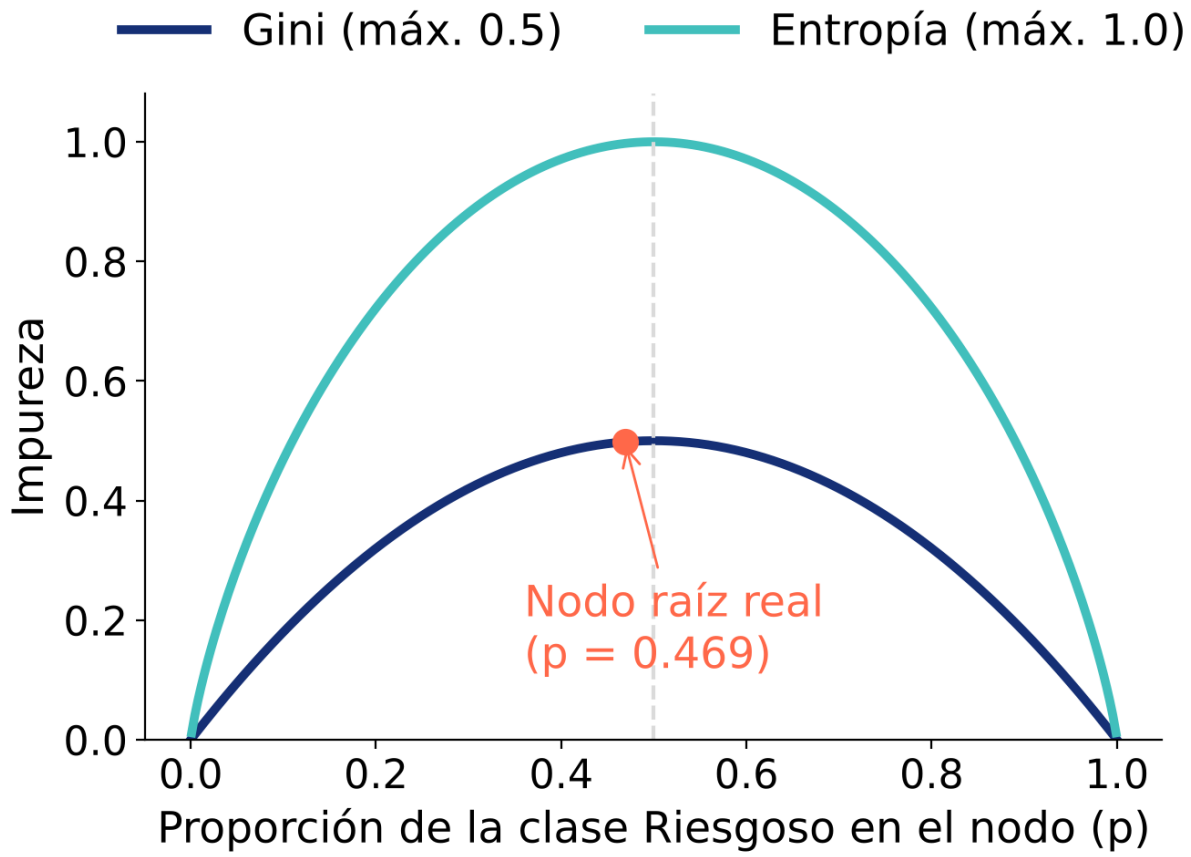
$$H(\text{raíz}) = -[\,0.531 \log_2(0.531) + 0.469 \log_2(0.469)\,] = 0.997$$

Rango (2 clases): H de 0 a 1 bit → más bajo es mejor · IG de 0 a H(padre) → más alta es mejor

Fuente: Shannon, C. E. (1948). A Mathematical Theory of Communication. Bell System Technical Journal.

Gini vs. Entropía

Dos formas de medir la misma idea: la mezcla de clases en un nodo



Comparación práctica

- Ambas valen 0 en un nodo puro y son máximas con 50 / 50.
- Máximo con 2 clases: Gini 0.5 · Entropía 1.0.
- Gini no usa logaritmos: es algo más rápida.
- Casi siempre eligen divisiones muy parecidas.

En scikit-learn: criterion='gini' (por defecto) o criterion='entropy'

¿Cómo Predice el Árbol un Caso Nuevo?

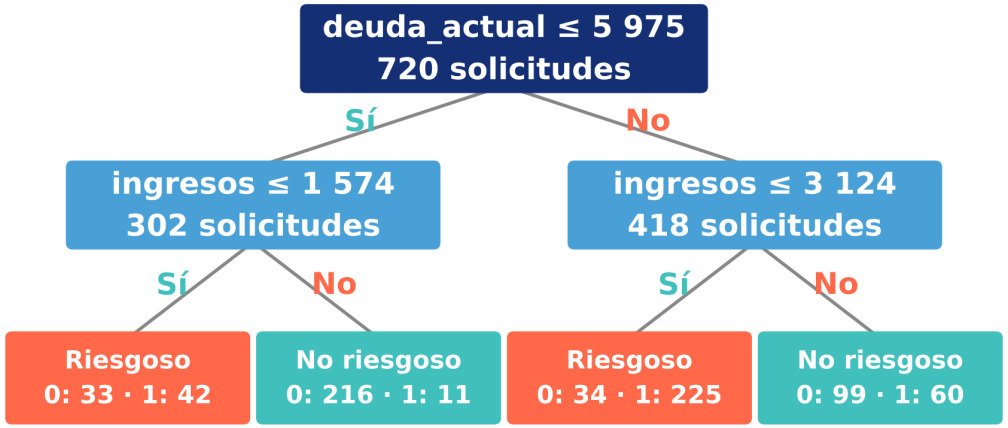
Árbol de profundidad 2 de «Anatomía del Árbol»: se recorre desde la raíz hasta una hoja

Solicitante (ilustrativo): deuda_actual = 4 000 · ingresos = 1 200

1 ¿deuda_actual ≤ 5 975? Sí → rama izquierda.

2 ¿ingresos ≤ 1 574? Sí → rama izquierda.

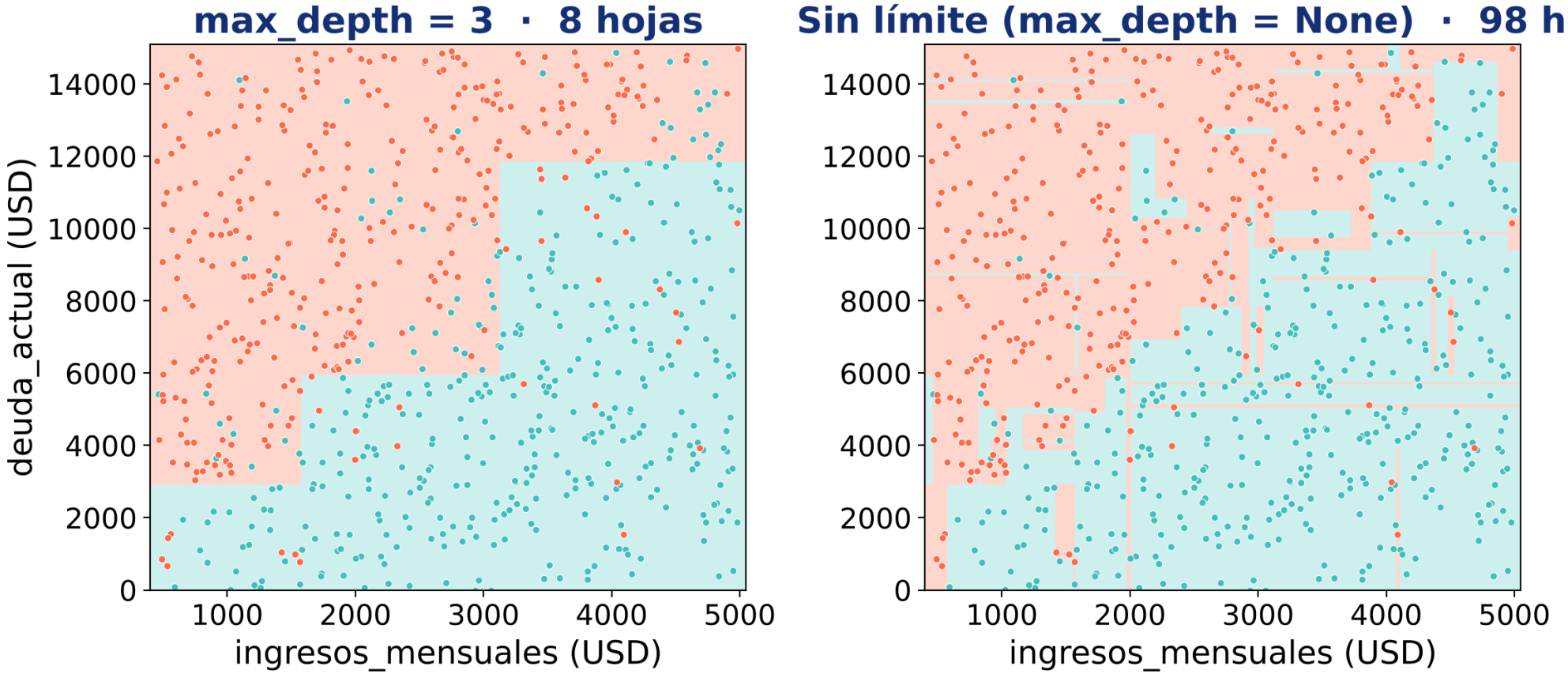
3 Hoja: 33 no riesgosos y 42 riesgosos → predice Riesgoso.



predict_proba(): devuelve la proporción de cada clase en la hoja: 42 / 75 = 0.56 de probabilidad de Riesgoso.

Sobreajuste: el Árbol que Memoriza

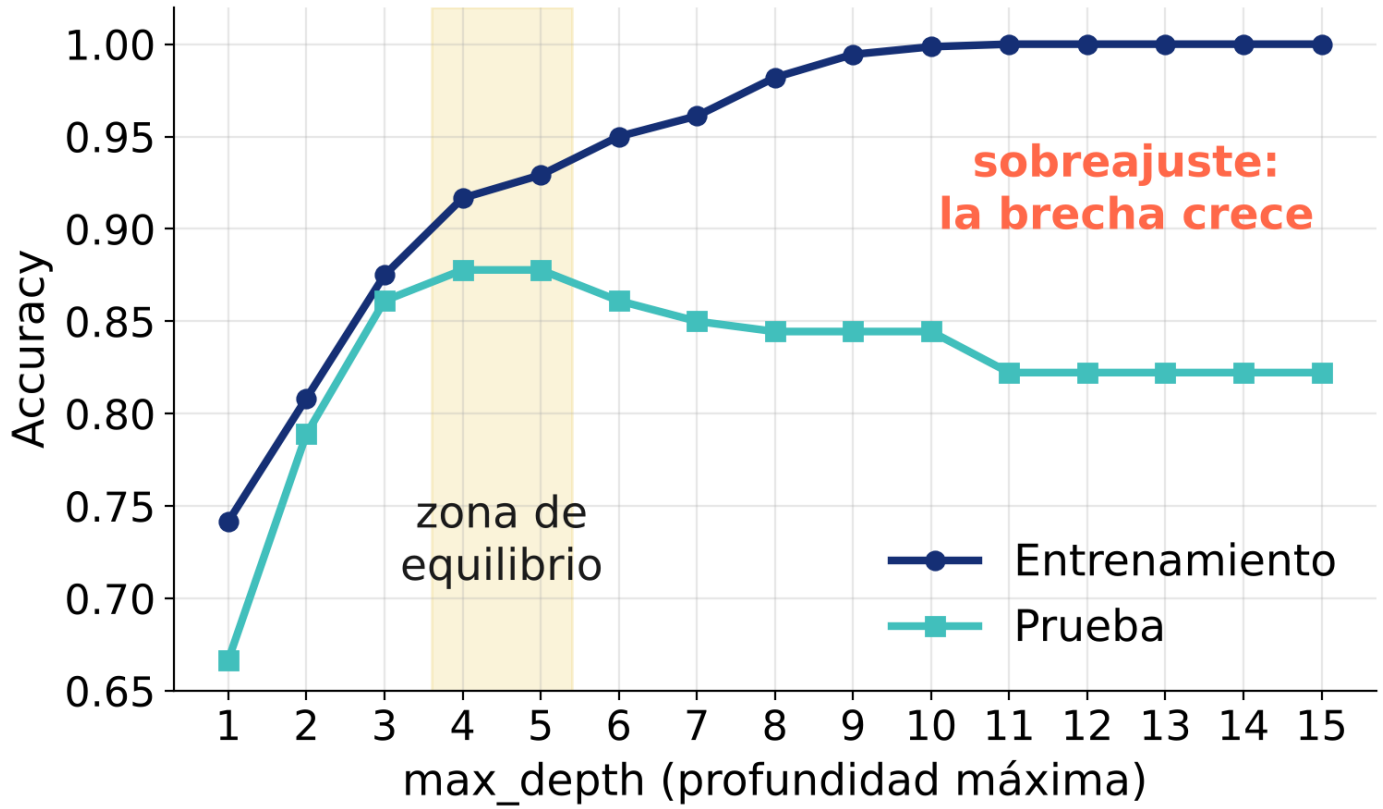
Solo 2 variables, para poder dibujarlo: por eso tiene más hojas que el árbol de 5 variables



Sin límite, el árbol crea regiones para casos aislados: aprende el ruido, no el patrón

Curva Real: Profundidad vs. Accuracy

Las 5 variables · Accuracy: rango 0 a 1 → 1 = deseado



Lectura

- Prof. 3: train 0.875 · test 0.861.
- Prof. 4–5: test 0.878 (mejor).
- Sin límite: train 1.00 · test 0.822.

Sobreajuste: el modelo aprende el ruido del entrenamiento y falla con datos nuevos. Señal: brecha grande entre train y test.

Hiperparámetros que Controlan el Crecimiento

Se fijan al crear el modelo, antes de fit(): «podan» el árbol por adelantado

Parámetro	Defecto	Qué controla	Efecto
`max_depth`	None (sin límite)	Niveles máximos de preguntas	Mayor → más complejo
`min_samples_split`	2	Casos mínimos para dividir	Mayor → más simple
`min_samples_leaf`	1	Casos mínimos en cada hoja	Mayor → más simple
`criterion`	'gini'	Medida de impureza	'gini' o 'entropy'
`random_state`	None	Semilla para desempates	Resultados repetibles

Con los valores por defecto no hay poda: el árbol crece hasta que sus hojas son puras o ya no pueden dividirse

La Clase DecisionTreeClassifier()

from sklearn.tree import DecisionTreeClassifier

```
from sklearn.tree import DecisionTreeClassifier

arbol = DecisionTreeClassifier(max_depth=3,      # poda
                               random_state=42)
arbol.fit(X_train, y_train)      # aprende las preguntas
y_pred = arbol.predict(X_test)   # recorre el árbol
```

Disponible solo después de fit()	Qué devuelve
`feature_importances_`	Importancia de cada variable (suman 1)
`get_depth()`	Profundidad real alcanzada
`get_n_leaves()`	Número de hojas
`classes_`	Clases aprendidas: [0, 1]

¿Hay que Escalar los Datos para un Árbol?

Pregunta frecuente después de SVM y Regresión Logística

Escalado: no es necesario

- Cada pregunta compara con un umbral: ¿deuda ≤ 5 975?
- Escalar cambia los números, pero no el orden de los casos.
- Las divisiones y predicciones no cambian.

Codificación: sí es necesaria

- scikit-learn solo acepta números en X.
- Texto (ej. motivo_consulta) → `pd.get_dummies`.
- Este dataset de crédito ya es 100% numérico.

⚠ **Importante:** StandardScaler no mejora un árbol de decisión. Por eso en esta clase no se usa un Pipeline con escalador: el árbol se entrena directo con X_train.

Dataset del Taller — Riesgo Crediticio

aprobacion_credito_ecuador.csv · 900 solicitudes · dataset sintético · sin valores nulos

Variable	Tipo	Descripción
edad	Numérica	Edad del solicitante (22 a 65 años)
ingresos_mensuales	Numérica	Ingresos en USD (452 a 4 996)
meses_empleo	Numérica	Meses en el empleo actual (1 a 240)
deuda_actual	Numérica	Deuda total actual en USD (0 a 14 992)
num_creditos_previos	Numérica	Créditos anteriores (0 a 8)
riesgo_crediticio	Objetivo (y)	1 = riesgoso · 0 = no riesgoso

Balance de clases

- No riesgoso: 478 (53%) · Riesgoso: 422 (47%)

Métricas

- Clases balanceadas → Accuracy y F1

Flujo Completo (1/3): Datos y Primer Árbol

Construir X, y, separar el conjunto de prueba y entrenar

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier

df = pd.read_csv('aprobacion_credito_ecuador.csv')
X = df.drop('riesgo_crediticio', axis=1)  # 5 variables
y = df['riesgo_crediticio']               # 0 o 1

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

arbol = DecisionTreeClassifier(max_depth=3, random_state=42)
arbol.fit(X_train, y_train)   # sin escalar
```

Flujo Completo (2/3): Evaluar el Árbol

Comparar el accuracy en entrenamiento y en prueba revela el sobreajuste

```python
from sklearn.metrics import classification_report

y_pred = arbol.predict(X_test)
print('Train:', arbol.score(X_train, y_train))  # 0.875
print('Test: ', arbol.score(X_test, y_test))    # 0.861
print(classification_report(y_test, y_pred))
```

Modelo	Hojas	Accuracy train	Accuracy test	F1 (Riesgoso)
max_depth=3	8	0.875	0.861	0.855
Sin límite	85	1.000	0.822	0.812

- Rango 0 a 1 → 1 = deseado. El árbol sin límite acierta todo en train y empeora en test.

Flujo Completo (3/3): Visualizar y Leer las Reglas

plot_tree() dibuja el árbol · export_text() lo escribe como reglas

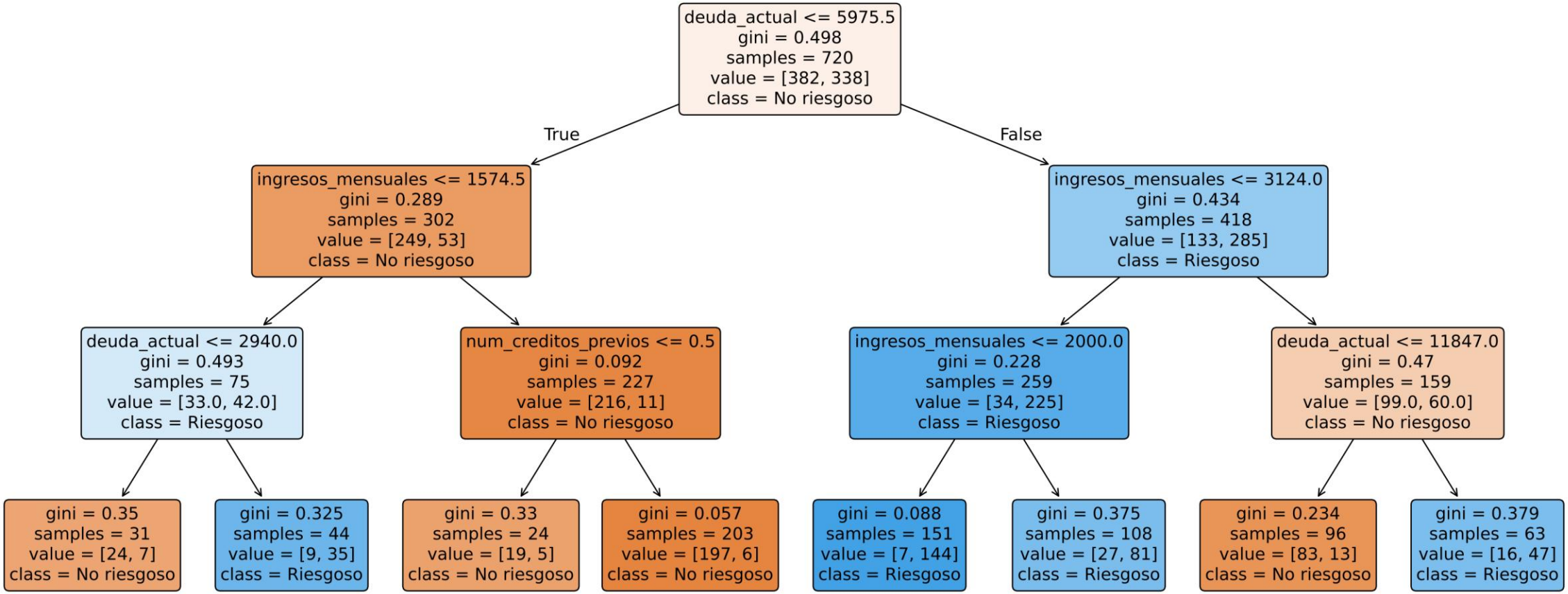
```
import matplotlib.pyplot as plt
from sklearn.tree import plot_tree, export_text

plt.figure(figsize=(20, 8))
plot_tree(arbol, filled=True,
          feature_names=list(X.columns),
          class_names=['No riesgoso', 'Riesgoso'])
plt.show()
print(export_text(arbol, feature_names=list(X.columns)))
```

```
|--- deuda_actual <= 5975.50
|   |--- ingresos_mensuales <= 1574.50
|   |   |--- deuda_actual <= 2940.00
|   |   |   |--- class: 0
```

Resultado de plot_tree()

Cada nodo: pregunta · gini · samples · value = [no riesgoso, riesgoso] · class

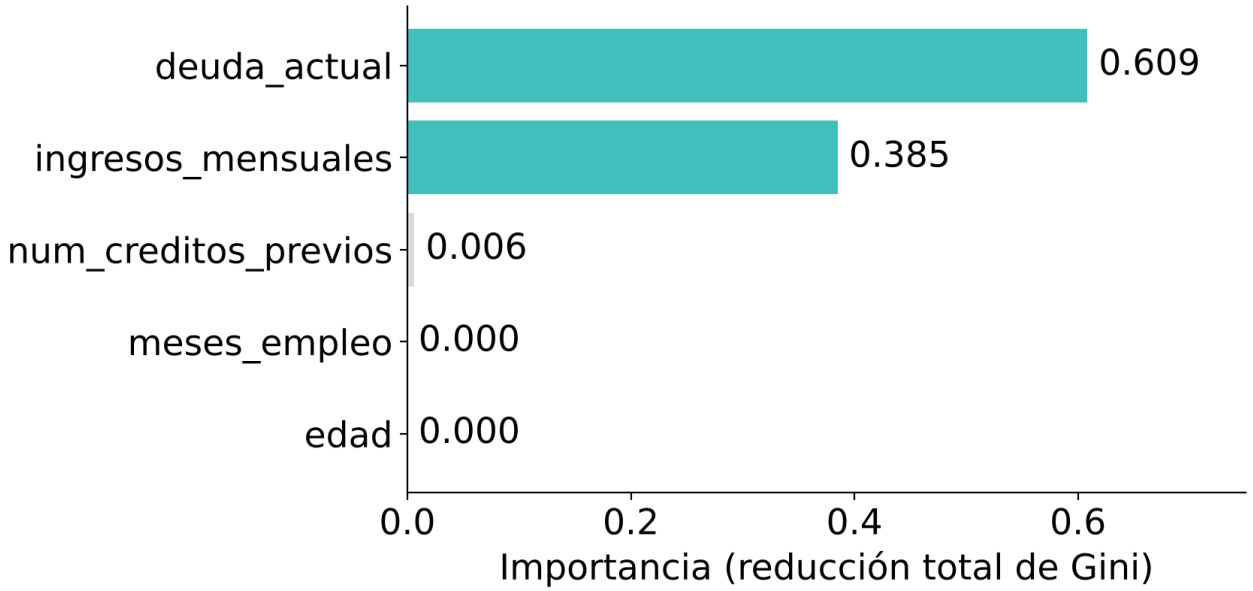


Colores de scikit-learn (igual que en Colab): naranja = No riesgoso · azul = Riesgoso · más intenso = más puro

Importancia de Variables: feature_importances_

Cuánto reduce cada variable la impureza (Gini) en todo el árbol

```
imp = pd.Series(arbol.feature_importances_, index=X.columns)
print(imp.sort_values(ascending=False))
```



Lectura (max_depth=3)

- deuda_actual (0.609) e ingresos (0.385) deciden casi todo.
- edad y meses_empleo = 0: el árbol no las usó.
- Rango 0 a 1; la suma de todas es 1.

GridSearchCV Aplicado al Árbol

Sin Pipeline, los nombres van directos: 'max_depth' (no 'modelo__max_depth')

```
from sklearn.model_selection import GridSearchCV

param_grid = {'max_depth': [3, 4, 5, 6, None],
              'min_samples_leaf': [1, 5, 10, 20]}
base = DecisionTreeClassifier(random_state=42)
grid = GridSearchCV(base, param_grid, cv=5, scoring='f1')
grid.fit(X_train, y_train)   # 20 combinaciones x 5 folds
print(grid.best_params_)     # depth 5, leaf 10
```

Modelo	Hiperparámetros	Accuracy test	F1 test
Sin límite	por defecto	0.822	0.812
Elegido a mano	max_depth=3	0.861	0.855
GridSearchCV	max_depth=5, min_samples_leaf=10	0.872	0.862

Ventajas y Limitaciones

Cuándo conviene un árbol de decisión y qué cuidar

✔ Ventajas

- Interpretable: se lee como reglas SI … ENTONCES.
- No requiere escalar variables.
- Captura relaciones no lineales e interacciones.
- Entrena rápido y mide la importancia de variables.

✘ Limitaciones

- Sobreajusta fácilmente si no se poda.
- Inestable: datos un poco distintos → otro árbol.
- Fronteras en escalón (cortes paralelos a los ejes).
- Un solo árbol suele rendir menos que muchos árboles combinados.

Regla práctica: siempre limitar el crecimiento (max_depth, min_samples_leaf)

Entrenamiento Final con el 100% de los Datos

Elegidos los hiperparámetros, el modelo final se entrena con las 900 solicitudes

Modelo de evaluación (80%)

- grid: eligió y midió el modelo.

Modelo final (100%)

- Se entrena con X, y; se exporta.

```
print(grid.best_params_)   # mejores valores encontrados

arbol_final = DecisionTreeClassifier(max_depth=5,
                                     min_samples_leaf=10,
                                     random_state=42)
arbol_final.fit(X, y)   # X, y completos, no X_train
```

Observa: con el 100% de los datos la raíz pasa a ser deuda_actual ≤ 8 014 (antes ≤ 5 975). Es la inestabilidad del árbol: se copian los hiperparámetros, no el árbol.

Exportación con joblib y Uso con Datos Nuevos

Otro programa solo necesita el archivo .pkl; el árbol no necesita escalador

```
import joblib
joblib.dump(arbol_final, 'modelo_riesgo_arbol.pkl')

modelo = joblib.load('modelo_riesgo_arbol.pkl')
nuevos = pd.DataFrame({
    'edad': [35, 28, 50],
    'ingresos_mensuales': [3800, 1200, 2500],
    'meses_empleo': [60, 18, 120],
    'deuda_actual': [2500, 4000, 9000],
    'num_creditos_previos': [2, 1, 4]})
print(modelo.predict(nuevos))   # [0 1 1]
```

Cliente 1 → 0

- Deuda baja, ingreso alto.

Cliente 2 → 1

- Ingreso bajo: $1 200.

Cliente 3 → 1

- Deuda alta: $9 000.